Introducción a la Programación

Algoritmos y Estructuras de Datos I

Segundo cuatrimestre de 2026

Departamento de Computación - FCEyN - UBA

Guía 8: Pilas y colas

Guía 8 - Ejercicio 1.1

Comandos básicos de pilas:

```
from queue import LifoQueue as Pila

p: Pila[T] = Pila()      #crear pila de tipo T
p.put(elem) # apilar elemento
p.get() # desapilar
elemento: T = p.get() #guardar elemento desapilado
p.empty() # preguntar si esta vacia (True o False)
```

Guía 8 - Ejercicio 1.1

`problema generar_nros_al_azar` ($in\ cantidad : \mathbb{Z}$, $in\ desde : \mathbb{Z}$, $in\ hasta : \mathbb{Z}$) : Pila[$\mathbb{Z}$] {

`requiere:` $\{cantidad \geq 0\}$

`requiere:` $\{desde \leq hasta\}$

`asegura:` {El tamaño de $resultado$ es igual a $cantidad$}

`asegura:` {Todos los elementos de res son valores entre $desde$ y $hasta$ (ambos inclusive), seleccionados aleatoriamente con probabilidad uniforme.}

}

Para obtener números al azar pueden usar la función `random.randint`($<$ `desde` $>$, $<$ `hasta` $>$) importando previamente el módulo `random`

Guía 8 - Ejercicio 1.1

Elaboremos casos de test para la solución. ¿Qué debe cumplir la pila generada? ¿Qué casos de test podemos pensar?

Caso	**Entradas**	**Esperado**
La cantidad es 0	cantidad=0 desde=0 hasta=10	Pila vacía
La cantidad es 10	cantidad=10 desde=0 hasta=10	Pila con 10 elementos
El rango es desde 10 hasta 20	cantidad=10 desde=10 hasta=20	Pila con elementos entre 10 y 20

Guía 8 - Auxiliar para armar una pila desde una lista

`problema pila_desde_lista` ($in\ l : seq\langle T \rangle$) : Pila[$T$] {
  `requiere:` $\{|l| > 0\}$
  `asegura:` {El tamaño de res es igual a $|l|$}
  `asegura:` {Las apariciones de los elementos de l son las mismas que en res}
  `asegura:` {El orden de los elementos de res desde su base hasta su tope es el mismo que el de l de izquierda a derecha}
}

Para definir la función para una pila de datos de tipo T genérico pueden usar

```
from typing import TypeVar
T = TypeVar('T')
```

Guía 8 - Auxiliar para ver el contenido de una pila

`problema mostrar_pila` $(in\ p : Pila[\mathrm{T}])$ {
  `requiere:` $\{True\}$
  `asegura:` {El contenido de p se imprime por pantalla, elemento a elemento de tope a base}
}

Guía 8 - Auxiliar para ver el contenido de una pila

problema `mostrar_pila` ($in\ p : Pila[\mathrm{T}]$) {
  `requiere:` $\{True\}$
  `asegura:` {El contenido de p se imprime por pantalla, elemento a elemento de tope a base}
}

Funtamental: no olvidar recrear la pila/cola de entrada antes de terminar la función. Debe quedar tal cual estaba al comienzo.

Nota: luego pueden modificar estas dos funciones para adaptarlas al tipo Cola.

Guía 8 - Ejercicio 1.3

`problema buscar_el_maximo` $(in\ p : Pila[\mathbb{Z}]) : \mathbb{Z}\ \{$
  `requiere:` $\{p$ no está vacía$\}$
  `asegura:` $\{resultado$ es un elemento de $p\}$
  `asegura:` $\{resultado$ es mayor o igual a todos los elementos de $p\}$
$\}$

Guía 8 - Ejercicio 1.3

Elaboremos casos de test para la solución.

Caso	**Entradas**	**Esperado**
Pila con un solo elemento	Pila([7])	7
Pila con varios elementos	Pila([1,2,3,4,5])	5
Pila con números negativos	Pila([-1,-2,-3])	-1

¿Qué otros casos de test podríamos probar?

Guía 8 - Ejercicio 2.13

Bingo: un cartón de bingo contiene 12 números al azar en el rango $[0, 99]$.

1. implementar una función $armar_secuencia_de_bingo$
2. implementar una función $jugar_carton_de_bingo$

```
from queue import Queue as Cola
#importa Queue y le asigna el alias Cola

c: Cola[T] = Cola() #crear una cola de tipo T
c.put(elemento) # encolar elemento
c.get() # desencolar
elemento: T = c.get() #guardar el elemento desencolado
c.empty() # preguntar si esta vacia (True o False)
```

2.13.1

`problema armar_secuencia_de_bingo` () : $Cola[\mathbb{Z}]$ {
  `requiere:` {True}
  `asegura:` {$resultado$ solo contiene 100 números del 0 al 99 inclusive,
        sin repetidos}
  `asegura:` {Los números de $resultado$ están ordenados al azar}
}

Sugerencia: Para ordenar de forma aleatoria una lista *s*, pueden usar la función *shuffle* de la librería *random*. Por ejemplo, `random.shuffle(s)`

2.13.2

`problema jugar_carton_de_bingo`
$(in\ carton : seq\langle\mathbb{Z}\rangle, in\ bolillero : Cola[\mathbb{Z}]) : \mathbb{Z}$ {
  `requiere:` {$carton$ solo contiene 12 números, sin repetidos, con valores entre 0 y 99 inclusive}
  `requiere:` {$bolillero$ solo contiene 100 números del 0 al 99 inclusive, sin repetidos}
  `asegura:` {$resultado$ es la cantidad mínima de jugadas necesarias para que todos los números del $carton$ hayan salido del bolillero}
}

Guía 8 - Ejercicio 2.13.2

Elaboremos casos de test para la solución.

Caso	**Entradas**	**Esperado**
El jugador gana en las primeras 12 jugadas	carton=[0,...,11] bolillero=Cola([0,...,11])	12
El jugador gana en la última jugada	carton=[0,...,11] bolillero=Cola([49,...,0])	50

¿Qué otros casos de test podríamos probar?